Enhancing Scheduling Efficiency in Dynamic Edge-Cloud Computing Environments leveraging A3C Learning with Transformers

Abstract: Optimal resource allocation is essential for the finite availability of computational resources at every tier of cloud computing. Rule-based and Reinforcement Learning (RL) methods are frequently employed to address resource allocation challenges, but the conventional algorithms face difficulties in dynamic circumstances due to their limited adaptability and responsiveness. The Asynchronous Advantage Actor Critic (A3C) technique offers a contemporary approach by integrating a reward system to efficiently optimise task parameters. An innovative technique that combines deep reinforcement learning with A3C using transformers is proposed for efficiently handling the tasks that involve large datasets and complex interactions. Transformers can comprehend the relationships between data sequences and managing dynamic patterns. The proposed approach shows potential in optimising tasks scheduling across Edge-cloud computing systems by significantly decreasing energy consumption by 9.7%, a major decrease in SLA violations by 40%, average tsk completion time by 21.8%, and cost reduction by 8.3%.

Keywords: Cloud Computing, Transformers, Dynamic Scheduling, Deep Reinforcement Learning, Mean Response Time, Task Migration Time, Energy Consumption, A3C.

1 Introduction

In this ever-growing world of information, it is becoming progressively more difficult to process the data. The numbers are overwhelming, but they are not slowing down. It is said that 90% of world's information & data has been generated over the last two years (B. Marr, 2018). We are surrounded by data in every sphere of our lives. There is a need of making data-driven decisions and such decisions are crucial in any field. These facts increase the importance of processing the data quickly and efficiently in this dynamic world.

Data processing is a complex task, and it is executed in the form of different smaller, simpler tasks like data gathering, data computation, data representation, data interpretation etc. The need for handling of these tasks demands powerful hardware and capable software like automation tools, monitoring tools for computing pipelines (G.D.C. Rodrigues et al., 2016). The above-mentioned multiple tasks constitute a workflow altogether, with each task requiring multiple resources. All the tasks cannot be performed together because of limitations in the resources and processing power of the computational node. Also, all the tasks are not independent of each other. There can be some tasks (also called child tasks) that cannot be executed before some other tasks (parent tasks) are performed first. There may be some tasks that require the same resources at the same time. Identification of the number of resources required and what those resources are, the number of tasks, task execution, and allocation of resources to each task according to the requirement and dependency of tasks on each other to reduce execution and waiting time of jobs, the statuses of the jobs etc is managed by a workflow scheduler. A workflow scheduler works as a scheduling algorithm. A good scheduler is the one that learns and adapts according to the changing environments, workloads and tasks.

There are different types of workloads and different types of working environments for which different algorithms have been designed. These scheduling algorithms can be broadly categorized as heuristic, metaheuristic and hybrid scheduling algorithms. Yet there is a need of scheduling algorithm that is fast and efficient enough in solving a scheduling problem for various tasks (or processes) and resources since the existing algorithms only consider some basic information to describe the states. Thus, the current technologies, services & platforms are being created and modernized, and new technologies are also being researched and developed. One of such new-age technology is Artificial Neural Networks (ANN). A workflow can be described as graph with the nodes describing the processes and the edges describing the dependency of tasks on each other. It is basically a sequence of interconnected tasks. Workflow scheduling can be considered as an optimization problem where numerous tasks must be distributed across multiple computing resources. It is a well-known NP-complete problem. Scheduling is the method of distributing the valuable computing resources, to various processes, threads, tasks, and applications that need them. The computational nodes may be distributed, and tasks may have to be executed parallel to each other. Workflow scheduling is a big issue even today. Manual scheduling of workflows is a tedious task.

Workflow scheduling using neural networks has recently drawn massive attention due to the budding field of *'Artificial Intelligence'* and *'Artificial Neural Networks'*, the potential of Neural networks field and also because of the wide-range applications of workflow scheduling both in

Copyright © 2021 Inderscience Enterprises Ltd.

scientific field like automation and monitoring tools for computing pipelines and economic field like business process management. The greatest challenge is the discreteness of this problem since the specific category and configuration of input and output required is not known to us. Thus, Q-Learning principles for Reinforcement Learning (Markel Sanz Ausin, 2020) are also used for mapping the requests of the user (or tasks) with the required resources which are available at that point of time. But this referenced technique is suffering from problems like initial population sensitivity, estimation of tasks in advance, stopping at low-quality optima, etc. Our model aims to automate workflow scheduling using artificial neural networks which is vital in every industry, from scientific fields to business-oriented fields.

The primary objective of introducing an ANN and Reinforcement Learning based Workflow Scheduling Optimization in Distributed Environment (WfSo_ANRL) is to allocate a significant work (or task) to the computing node such that the amount of data transfer time, waiting time and the execution time of the job are minimal (B. Varghese et al., 2018). It is (Data transfer, waiting time and Execution time) collectively ensured by measuring the makespan time when different workflows are passed through the scheduler. The makespan is minimized by appropriately allocating the tasks to the computational nodes and selecting appropriate course of action of workflows.

The workflow scheduling problem cannot be solved completely owning to its NP-Complete Nature. However, the best possible scheduling can be achieved based on target parameters. We tried to address this issue by scheduling the tasks on the nodes to minimise the makespan on the resulting schedule. The proposed method executes well in a real-time environment with some time and budget constraints. However, the proposed solution forms a set of parameters encoding the state of the environment. As workflows and environment may differ in input data and scale, the algorithm operates with relative values. This assumes that the proposed solution would perform dynamic scheduling of heterogeneous computational load. Dynamic scheduling is suggested as a solution to the increasing use of cloud-based apps. These applications require access to data and require the execution of actions that are in line with certain objectives. Deep reinforcement learning is recommended for supporting these tasks, albeit it requires sufficient computational infrastructure to meet user needs (McKinsey & Company, 2018). The popularity of deep reinforcement learning is driven by the need to improve network latency, reduce costs, and optimise resource utilisation. Conventional cloud-based deployments of IoT applications often fail to provide timely replies, especially for time-sensitive applications like healthcare, emergency response, and traffic monitoring in IoT applications. Edge and cloud computing offer benefits through their ability to minimise latency (J. Wang et al., 2019). Optimal allocation of resources is crucial for computational operations at the edge level. However, the act of scheduling at the edge node presents difficulties as a result of the constraints on available resources. In such cases, a model is essential for accurately allocating jobs according to their resource utilisation. Our goal is to create a model that can identify tasks, evaluate their resource needs, and allocate them to hosts. The remaining jobs will be transferred to different hosts depending on their available resources.

This solution implements the utilisation of A3C, a reinforcement learning model, in conjunction with the Transformers neural network. The policy used is consistently revised and rectified within the Resource Management System (RMS), where work assignments occur. Nevertheless, the cloud-based computing network may experience instances of task execution failure, resulting in a compromise of the Quality of Service (QoS) due to resource limitations. Scheduling tasks in a cloud environment is difficult because of the unpredictable nature of server resources, such as response time, capacity, RAM, different speeds, energy usage, and the intermediary layer between the edge and cloud network. Traditional scheduling techniques such as First-ComeFirst-Served (FCFS) and Round Robin, which follow fixed rules, face difficulties in handling changing workloads, leading to inefficient use of resources and limited flexibility (T.Choudhri et al., 2018). Workload spikes' unpredictability can result in suboptimal utilisation of server resources (X. Chen, 2015).In order to address these difficulties, it is crucial to use a continuous and dynamic approach that considers the efficient utilisation of multiple resources at different levels. This method aims to improve the quality of service (QoS) while minimising energy and resource expenses (D. Basu et al., 2019). Reinforcement Learning, a method of making decisions by interacting with the environment, is chosen to tackle these dynamic scheduling challenges (D. Jeff et al., 2018, 2019). Dynamic scheduling, which entails the real-time adjustment and optimisation of resource allocation, is in line with the concepts of reinforcement learning. The Reinforcement Learning is driven by the intricate nature of decision spaces in dynamic scheduling issues. In such problems, agents learn by experimenting and making adjustments in the face of uncertainties, with the goal of improving resource utilisation over time (A. Beloglazov et al., 2012). This technique employs both the Resource Management System (RMS) (M. Xu et al, 2017) and neural networks in a deliberate manner. Research has shown that value-based reinforcement learning tends to perform poorly in settings with large levels of randomness (V. Mnih et al, 2015).

The suggested solution utilises Transformers, a type of deep learning model, to address scheduling challenges. It attempts to improve the Quality of Service (QoS) and handle temporal trends in workload. This is achieved by implementing asynchronous policy gradient methods within a decentralised environment (V. Mnih et al, 2015). In order to tackle this difficulty, we have opted for a scheduling model that focuses on deep policy gradients. This model has the ability to comprehend the variety of resources and the intricate dynamics of workloads. We employ the Asynchronous Advantage Actor Critic, an asynchronous policy-gradient reinforcement learning technique, to effectively adapt the model to changing situations (A3C) (V. Mnih et al, 2015). The A3C approach, initially proposed by (Minh et al) (V. Mnih et al, 2015). It has been especially utilised in dynamic scheduling. It has been very effective when combined with the Residual Recurrent Neural Network (R2N2). Within our specific construction, we integrate the A3C algorithm with

the Transformers neural network. Transformers utilise parallel processing in their neural networks, enabling them to analyse input sequences concurrently. This leads to faster training compared to the sequential processing used in recurrent models. A3C is a policy gradient approach that iteratively changes a stochastic policy by running many actor-agents in parallel, each with its own neural network. These agents receive simultaneous training, consistently refreshing the global network using specified parameter values. After every iteration, these agents restore their configurations to reflect the worldwide network and carry out their assessment and training routines. The continual updating process allows for quick adjustment to unpredictable environments, resulting in better results (V. Mnih et al, 2015). Within this framework, we incorporate a deep learning model known as Transformers, which was first presented by Minh et al. (Vaswani. A et al., 2017), into neural network. This is done in conjunction with our reinforcement learning model A3C. The utilisation of Transformers (Vaswani. A et al., 2017), has showcased adaptability and efficiency in diverse fields. The self-attention mechanism in Transformers allows for the detection and understanding of complex connections among various elements in a sequence, making them well-suited for jobs that involve dynamic and evolving patterns. This model has the capacity to acquire knowledge and adjust to changing scheduling patterns, interdependencies, and priorities. This system is capable of efficiently managing and responding to pertinent information during the scheduling process, making it very skilled at handling dynamic situations in which priorities and requirements may change over time. To limit loss and improve the quality-of-service metrics, we plan to adjust settings according to the changing environment.

The test uses a brand-new version of the iFogSim Resource Kit that works with CloudSim 5 (R. N. Calheiros, e al, 2011). Which improves how our model is used and saves a lot of energy.

The main objectives of this research encompass:

1. Optimize resource allocation: Effectively distribute resources in dynamic Edge-Cloud systems by utilising A3C learning in conjunction with Transformers.
2. Optimise latency: Improve the efficiency of scheduling in Edge-Cloud configurations to decrease response time.
3. Minimise energy consumption: Apply tactics to decrease energy usage while upholding performance levels in dynamic computing environments.
4. Minimise SLA breaches: Resolve and decrease occurrences of Service-Level-Agreement violations by implementing enhanced scheduling approaches.
5. Improve flexibility: Create a scheduling system that incorporates A3C and Transformers to adjust to changing workload patterns in Edge-Cloud scenarios.

The remaining components of the task have been organised in the subsequent sections:

Section 3 provides a detailed explanation of the system's operational model and the phrasing of the challenge. In our Deep Reinforcement Learning (DRL) model, Section 3.4 explores the implementation of a particular policy gradient. Section 3.8 demonstrates the utilisation of the Transformers neural network alongside the Asynchronous Advantage Actor-Critic (A3C) algorithm. The findings are further upon in Section 4, and the conclusions are presented in Section 5.

2 Literature Review

This literature review focuses on evaluating the efficacy of our existing model in comparison to other baseline models. Historically, scheduling algorithms have relied on rule-based approaches, which restricts their efficiency in handling dynamic workloads (A. Beloglazov et al, 2012). Recent studies have investigated value-based reinforcement learning techniques, with a specific emphasis on optimising parameters for resource management systems in remote environments. Nevertheless, table-based reinforcement learning has substantial scalability challenges when managing a big quantity of operating units. Furthermore, the ability to adjust to contexts characterised by a high degree of resource variability can have a significant impact on the performance of the model (M. Bowling, 2000). Additional inquiries revealed difficulties in validating and guaranteeing the efficacy of these models (SanathMugeraya and Kailas Devadkar). Another issue pertains to the limitations in the sequence of execution and fluctuations in communication duration, particularly in priority-driven processing (Lei Shi et al, 2021). An solution that utilised a directed acyclic graph (DAG) faced challenges in accurately describing and adjusting to dynamic and intricate situations (Lei Shi et al, 2021).

This work focuses on the core issue of resource allocation and job scheduling in edge computing using Deep Reinforcement Learning (DRL). This strategy entails creating a scheduling policy that allocates task components of application tasks to geographically dispersed edge nodes, with the main objective of minimising the total or average application delay inside the system. The system model comprises an edge computing environment with geographically dispersed edge nodes connected via a communication network. Additionally, it incorporates a cloud-based facility for aggregating data processing. This model integrates a Deep Reinforcement Learning (DRL) framework, which sets it apart from typical scheduling approaches. The edge unit node consists of a group of computing hosts, which can be either centralised at a single location or spread out within a certain area. The communication network facilitates uninterrupted data transmission between the edge nodes. The study utilises a dataset consisting of both artificially created data simulation and real-world data from the Google Cloud dataset for the present assessment (Z. Cao et al, 2019). The application scenario entails the process of arranging and allocating a set of components to an application project. These components may require processing at several geographically scattered edge nodes, resulting in the introduction of limitations on the sequence of execution among them.

The talk centres on the difficulties associated with effectively scheduling components in Cloud computing and offers a summary of its developmental chronology. This research conducts a comparative analysis of several scheduling methods and optimisation objectives in the context of Cloud

computing, emphasising the significance of resource management and job distribution. Significant focus is given to the utilisation of machine learning, particularly Deep Reinforcement Learning (DRL), to tackle these scheduling difficulties.

The study primarily focuses on the Cloud computing environment, which is defined as a network of distributed computers connected by high-speed networks (Z. Cao et al, 2019). This environment encompasses internet-based applications that are delivered as services, where both the hardware and system software dynamically provide services to consumers. The study explores the utilisation of Deep Reinforcement Learning (DRL) for the allocation of resources in Cloud computing. Deep Reinforcement Learning (DRL) is considered an ideal approach for scheduling in Cloud computing due to its integration of deep neural networks and reinforcement learning. This combination effectively tackles the intricate scheduling challenges that arise in this domain. The inherent stochasticity in electrical components, coupled with the unpredictability of user requests, poses substantial difficulties in constructing DRL models that can effectively respond to these uncertainties. Conventional scheduling methods frequently encounter difficulties in efficiently handling the unpredictable characteristics of Cloud computing. Creating Deep Reinforcement Learning (DRL) algorithms capable of acquiring knowledge and adjusting to unpredictable circumstances is a challenging endeavour. The scheduling problem in Cloud computing is widely recognised as NP-complete or NP-hard, which contributes to its computational complexity. Applying Deep Reinforcement Learning (DRL) algorithms to address these intricate issues necessitates meticulous evaluation of computational resources, training durations, and the overall efficacy of the learning process. The issue lies in finding a balance between the requirement for precise scheduling decisions and the significant computing expenses involved.

The incorporation of DRL-based scheduling into pre-existing Cloud infrastructure has its own distinct set of issues. It is crucial to guarantee compatibility with existing scheduling algorithms. Ensuring minimal interruptions to current services is another crucial factor to consider. Furthermore, it is essential to deal with potential conflicts that may arise with defined protocols throughout the implementation phase.

By conducting a comprehensive analysis of current literature, we have found numerous obstacles in the process of scheduling jobs inside edge-cloud setups (Guangyao Zhou et al). Based on the findings of these investigations, our goal is to enhance our model by integrating Transformers with A3C (Asynchronous Advantage Actor-Critic) learning in a substantial manner. The objective of this research is to assess and contrast the effectiveness of our existing model in comparison to other standard models. Conventional scheduling algorithms, which depend on rule-based methods, are insufficient for handling dynamic workloads. Recent research has investigated the use of value-based reinforcement learning algorithms to optimise parameters for resource management in distributed systems. Nevertheless, table-based reinforcement learning encounters constraints such as scalability challenges and struggles to adapt to contexts characterised by significant resource unpredictability. The efficacy of our present strategy is underscored as it effectively tackles and surmounts these problems. Furthermore, we recognise the need of addressing concerns regarding the verification and efficiency of the microservice architecture. Although the use of a directed acyclic graph (DAG) method has been suggested, there are issues over its capacity to accurately depict and adjust to dynamic and intricate situations. This paper presents our approach as a highly promising option for addressing the highlighted issues in scheduling work in edge-cloud scenarios.

This literature review focuses on the challenges associated with attaining optimal Multi-Objective Task Scheduling (MOTS) in large-scale Infrastructure-as-a-Service (IaaS) Cloud Computing (CC) environments (R. Nithiavathy). The article presents a novel approach called the Slime Mould Algorithm based on Adaptive Guided Differential Evolution (AGDESMA). AGDESMA is designed to achieve a harmonious equilibrium between the exploration and exploitation stages by harnessing the characteristics of the Slime Mould Algorithm (SMA) and Adaptive Guided Differential Evolution (AGDE). AGDESMA's objective is to mitigate premature convergence by promoting a wider range of individuals within the population and improving the efficiency of local search within the group. Through rigorous experimental validation using CloudSim, AGDESMA has been proven to outperform benchmark systems, especially in terms of computational overhead and the balance between makespan and financial cost. This validation was conducted utilising both generated and conventional workload traces. The findings confirm the effectiveness of AGDESMA in dealing with the intricacy of optimising large-scale MOTS in IaaS CC systems. This emphasises its capacity to surmount obstacles in rapidly changing cloud settings. It is suggested that future study should focus on improving AGDESMA's ability to handle larger amounts of data, its flexibility to adapt to multiple cloud deployment methods, and its usefulness for a variety of applications. This will aid in the continuous endeavours to enhance work scheduling in cloud computing. Nevertheless, the study fails to specifically discuss the algorithm's capacity to react to dynamic changes in the cloud environment, such as variations in workload demand. It is essential to maintain the effectiveness of AGDESMA in the face of constantly changing situations to ensure its practical implementation. This literature review focuses on the importance of doing experimental validation using CloudSim and synthetic workload traces. Nevertheless, the lack of validation in practical cloud environments, where real workloads and infrastructure are present, creates a potential disparity between simulation outcomes and actual performance in real-world scenarios.

This research project focuses on solving the intricate challenge of scheduling workflows on heterogeneous cloud resources, considering market dynamics and schedule constraints (Chang Lu et al. 2023). The paper presents a Multi-Hierarchy Particle Swarm Optimisation (MHPSO) algorithm that is guided by heuristic and meta-heuristic principles. The main objective is to minimise the total

financial expenses related to on-demand pricing models, while simultaneously considering the time delays in transmitting data across interdependent processes on different virtual machines (VMs). The MHPSO proposal consists of three primary elements: a pre-processing workflow aggregation method, a particle production system for creating an initial population, and a hierarchical evolving process for iterative evolution. An analytical validation of the parameters and components is performed across many workflows, and a comparison analysis is conducted to compare them with existing approaches. The research focuses on addressing issues associated with the diversity of virtual machines, dependencies on execution time, and on-demand pricing systems. It presents a new hybrid strategy that combines both efficiency and effectiveness. The main contributions consist of a scalable system for aggregating workflows, a technique for dynamic expansion, and the MHPSO algorithm that incorporates a distinctive evolutionary process. The simulation trials demonstrate the algorithm's efficacy in generating high-quality solutions with efficiency. The study examines the contributions, organisational factors, and potential areas for future research, with a focus on the significance of resolving workflow scheduling problems in cloud systems. Nevertheless, the article presupposes the utilisation of a directed acyclic graph (DAG) as the representation for workflows. Although this depiction is frequently used in the literature on workflow scheduling, it may not accurately encompass the various types of workflow architectures found in real-world situations. The algorithm's performance may be affected by the attributes of the processes to which it is applied. While the research does compare the proposed MHPSO algorithm to other approaches, it does not specifically evaluate its performance against the most advanced meta-heuristics for workflow scheduling. Expanding the investigation to incorporate a wider array of algorithms would yield a more thorough assessment of MHPSO's capabilities and limitations.

The efficiency of data centres is of paramount importance in the field of cloud computing, and the optimisation of energy consumption is a significant challenge (Ding et al., 2024). An essential strategy for attaining energy efficiency entails the strategic allocation of virtual machines (VMs). Genetic algorithms (GAs) are a prominent method for addressing the VM placement problem because they can produce solutions of exceptional quality. Nevertheless, the high processing requirements of conventional genetic algorithms have impeded progress in improving energy efficiency, particularly in scenarios that necessitate quick and lightweight solutions. This work introduces a new method called the Accelerated Genetic Algorithm with Population Control. This approach integrates a flexible population size strategy and is distinguished by the enhancement of gene diversity, adaptive mutation rates, and expedited termination criteria. The Accelerated Genetic Algorithm with Population Control presents a promising approach to provide energy-efficient virtual machine (VM) placement in large-scale data centres. It provides a trade-off between computational efficiency and solution quality for the expansion of energy-conscious computing infrastructures.

The effectiveness of the algorithm relies on accurately adjusting parameters, such as mutation rates and termination criteria. This study investigates the difficulties linked to auto-scaling techniques in the Knative serverless platform, a well-known choice for serverless computing (Minh-Ngoc et al., 2023). Although Knative's Horizontal Pod Autoscaler (K-HPA) provides dynamic resource allocation based on current traffic, it has limitations in promptly responding to abrupt increases in demand and ensuring optimal resource utilisation. Current hybrid auto-scaling solutions typically prioritise either horizontal or vertical scaling, whereas reinforcement learning-based methods are intricate and challenging to incorporate into the Knative platform. This study presents a new and innovative hybrid auto-scaling approach that utilises Kubernetes operators and bespoke resources, without requiring any modifications to the Knative environment. The system employs traffic forecasting to optimise allocated resources and concurrency levels for optimal performance. The suggested method presents a hopeful direction for improving the auto-scaling effectiveness of Knative, which is in line with the growing requirements of serverless computing in practical applications. The study highlights the significance of prioritising latency Service-Level Objectives (SLOs) and offers valuable insights into the practical implementation of the suggested solution in different situations. Nevertheless, the efficiency of the system is greatly dependent on the precise prediction of future service request traffic. If there is not enough previous traffic data or if it does not adequately reflect the dynamic workload patterns, the system's predictive capabilities may be hampered.

This paper investigates the intricacies of task offloading in Internet of Things (IoT) contexts, specifically examining the difficulties presented by Mobile Edge Computing (MEC) and issues over data insecurity (Moghaddasi et al.,2023). To tackle these concerns and attain a harmonious equilibrium between energy consumption, expenses, and time delay, the study introduces an innovative offloading system that utilises blockchain technology. This framework combines a cutting-edge Double Deep-Q-Network (DDQN), a Deep Reinforcement-Learning (DRL), the Technique for Order of Preference by Similarity to Ideal Solution (TOPSIS), a Multi-Criteria-Decision-Making (MCDM) technique and a Multi-Criteria Decision-Making (MCDM) tool. The MEC-based offloading problem is formulated as a Markov Decision Process (MDP) in this technique, enabling the DDQN to acquire the most effective strategy. Afterwards, the TOPSIS method is utilised to determine offloading decisions according to predetermined parameters.

The results illustrate the efficacy and resilience of the suggested framework, providing a substantial contribution to tackling the intricate problems linked to task offloading in IoT environments. Nevertheless, a possible disadvantage of the framework is its dependence on precise and current data recorded in the blockchain. The efficacy of the offloading decisions relies significantly on the quality and promptness of the data documented on the blockchain.

This literature review explores the difficulties surrounding the allocation of virtual machines in cloud data centres, with

a focus on the growing energy consumption and the importance of enhancing energy efficiency (Hamid Reza Naji et al). The suggested method presents a new technique that combines learning automata with a multi-objective cuckoo search optimisation algorithm, called LAMOCS (Learning Automata-based Multi-Objective Cuckoo Search). The strategy seeks to achieve load balancing, energy efficiency, and resource optimisation in cloud service providers. The simulation findings clearly indicate that the method consistently outperforms over other approaches, making it a highly attractive alternative for optimising energy efficiency and resource utilisation in cloud computing systems. The research also proposes potential applications of this approach for resource allocation difficulties in distributed computing systems, highlighting areas for future investigation.

This literature review examines the important topic of Real-time System Reliability Monitoring (RTRM) in the context of edge computing, which is known for its diverse and decentralised nature (M. J. Divan et al., 2023). The evaluation encompasses a Systematic Mapping Study (SMS) that investigates 50 pertinent papers published between 2013 and 2022. The investigation uncovers an increasing fascination with Real-Time Risk Management (RTRM), particularly in the fields of networking and health. This is particularly evident in the utilisation of RTRM in body sensor networks and the management of data policies in edge computing environments. The study emphasises the early stage of Real-Time Risk Management (RTRM) in edge computing, pointing out the absence of established guidelines and a strong focus on push-based data collection methods for central decision-making. In addition to the networking and health sectors, industrial and environmental monitoring are identified as important applications. The assessment highlights significant obstacles and prospects for progress, including issues with data compatibility, decentralised decision-making models, standardisation of experimental design, control over data ownership, effective utilisation of organisational knowledge, and development of calibration techniques for data sources. This literature review offers significant insights into the current status of RTRM in edge computing through a methodical and comprehensive approach. It establishes a basis for future research efforts in these rapidly evolving technologies.

This literature review specifically examines the difficulties related to job migration in Edge Computing (EC), specifically when mobile devices (MDs) move outside the range of an edge server (ES) (Zhenyu Shi et al.). Contemporary research frequently neglects the connections between different tasks and the presence of uncertainty in computing environments. The study presents the I-MaOWMUE model, which is a new method that converts uncertain elements into interval parameters for processing, to enhance the efficiency of transferring work processes in unpredictable environments. The model integrates function objectives such as maximum completion time, migration delay, load balancing, and energy consumption. Additionally, it employs real-time priority scheduling algorithms.

The research proposes the Migration-Based Interval Many-Objective Evolutionary Algorithm (MI-MaOEA) to address situations when there are numerous MDs and ESs involved in migration. This algorithm takes into consideration that the task dependencies and variations in ES load. The simulation results clearly illustrate the superior optimisation capabilities of MI-MaOEA in comparison to other approaches. The paper situates its contribution within the European Commission (EC) framework, highlighting the necessity for complete migration strategies in situations that involve multiple Mobile Devices (MDs) and Edge Servers (ESs). The paper encompasses preliminary research, characteristics of the I-MaOWMUE model, specifics of the MI-MaOEA algorithm that has been suggested, results of the simulations, and final remarks. Implementing such intricate models in practice may necessitate significant computational resources and technical proficiency, presenting difficulties for real-world use, especially in situations with limited resources. Furthermore, although the simulation results demonstrate the effectiveness of MI-MaOEA, they may not completely reflect the dynamic and unpredictable characteristics of real-world EC environments.

The literature reviewed in this section highlights specific constraints that necessitate consideration in the current study are as follows:

1. Conventional IoT setups that rely on cloud computing have difficulties due to increased network latency, leading to delayed answers for time-critical applications including healthcare, emergency response, and traffic enforcement.
2. Edge nodes have limited processing resources due to cost and feasibility limitations, highlighting the significance of optimising resource utilisation.
3. Tabular value-based Reinforcement Learning (RL) algorithms face challenges in terms of scalability when dealing with highly unexpected events.
4. Deep Q Learning (DQN), a type of value-based reinforcement learning (RL) that is which is not suitable for extremely unpredictable conditions. As a result, it does not perform well in Edge-Cloud deployments.
5. Existing research fails to sufficiently explore the potential of utilising temporary patterns in network, workload, and behaviour of nodes to enhance decisions about the allocation of time and resources
6. Present study depends on centralised scheduling rules that may not be optimum for distributed or hierarchical instance.
7. Most articles do not examine asynchronous learning models and policy gradients in their research.

In the next part, we will provide a detailed explanation of the operational model for implementing Deep Reinforcement Learning (DRL) methodologies, with a specific focus on the Asynchronous Advantage Actor-Critic (A3C) algorithm. Our approach involves using transformer-based models to schedule Cloud computing resources. The goal is to accurately represent the intricate linkages and patterns in the ever-changing Cloud environment. Transformers, renowned for their efficacy in natural language processing tasks, are specifically tailored to comprehend and depict the complex

interdependencies that exist in Cloud scheduling scenarios. The A3C algorithm is used as an asynchronous and distributed reinforcement learning method to address the difficulties of real-time adaptation and scalability in the Cloud domain.

The implementation approach include training the A3C model using transformer topologies on comprehensive datasets that encompass diverse Cloud scenarios, user needs, and resource utilisation patterns. The dataset integration encompasses information on CPU, GPU, RAM, disc capacity, and network bandwidth usage over time, capturing both the temporal and spatial components that are crucial for making efficient scheduling decisions. When integrating A3C with transformers for Cloud scheduling, it is important to carefully analyse the complexity of the algorithm, the interpretability of the results, and the ability to handle the uncertainty and randomness of electronic components in the Cloud environment. Implementing this strategy effectively has the potential to optimise resource scheduling in Cloud computing, resulting in greater performance, decreased energy usage, and improved user happiness. CloudSim will be utilised throughout the implementation phase. CloudSim facilitates the generation of simulated cloud environments, enabling users to imitate the actions of various cloud elements such as data centres, virtual machines, and workloads. The simulation environment is essential for evaluating algorithms, tactics, and policies without the need for an actual cloud infrastructure.

3. System Design and Method

This section provides a detailed overview of the operational system. The text begins by introducing CloudSim and other relevant technologies, with a focus on the simulation environment. The conversation shifts to the dataset used and the consequent framing of the problem. An analysis is conducted on the neural network architecture, offering a comprehensive explanation of its structural composition. The section closes by providing a systematic comprehension of the entire process, explaining the essential components and their functions in facilitating the implementation of the model.

3.1 System Model

The integrated arrangement employs a combination of edge and cloud nodes to create a network hierarchy. This hierarchy includes decentralised heterogeneous resources that span from the network's edge to the complex and interconnected remote cloud infrastructure. These computer functions serve as hosts for different functionalities and display significant differences in computational capacity and reaction times. Edge devices, located near users, offer considerably faster response times but are constrained by their computational capacities. However, cloud resources, which include virtual machines and are located at several layers separated from users, result in longer response times. The cloud nodes provide abundant resources, showcasing sophisticated computing skills that can efficiently handle several jobs simultaneously.

The framework illustrated in Figure 1 comprises several components, namely the Deep reinforcement learning system, task arrangement and relocation of services, a service for monitoring resources, and a module for solving constraint satisfaction problems. The Resource Monitoring Service (RMS) operates by receiving tasks along with their specified quality-of-service requirements. The RMS ensures that every task is completed, and if a task cannot be conducted, it may transfer the work to another host that meets its resource requirements.

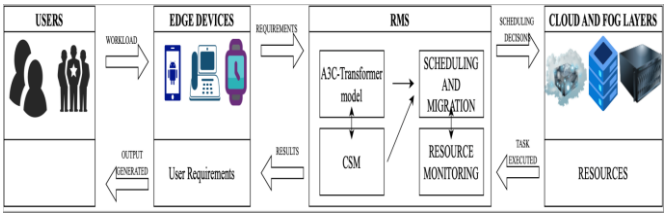

Fig .1 System working model

The coordination between the services for scheduling and migrating tasks, entails engaging with a deep reinforcement learning module (DRLM), that offers allocation suggestions for every task assigned to hosts, these services are performed. Rather than relying on a single scheduler, many schedulers are implemented, with each one assigned specific task divisions and nodes. Through the utilisation of Asynchronous Advantage Actor-Critic (A3C), we deploy several actors to learn parameter updates asynchronously, hence dispersing the computational load over different hosts. This technique enables rapid learning while working within the limitations of edge devices with minimal resources. Our method operates under the assumption that both edge and cloud nodes collect and merge gradient values within a specific region with their respective nodes.

The gradients, which come from separate schedulers, are subsequently merged and synchronised across all hosts to facilitate the autonomous updating of their models. Within our policy learning technique, a crucial element of deep reinforcement learning, each scheduler maintains an individual version of the deployed neural network to facilitate asynchronous upgrades. The Constraint Satisfaction Module (CSM) is an essential element of the Resource Monitoring Service (RMS). The CSM is responsible for verifying the recommendations made by the deep reinforcement learning model about constraints. These constraints encompass factors such as the ongoing migration of a task or the target host running close to its maximum capacity.

3.2 Workload model

The task generation process employs a stochastic approach, where each task is assigned dynamic costs. Over time, the processing and bandwidth requirements of jobs rise due to the escalating user demands and the mobility of IoT devices. Our approach involves dividing the total time used for execution into equal-duration scheduling periods.

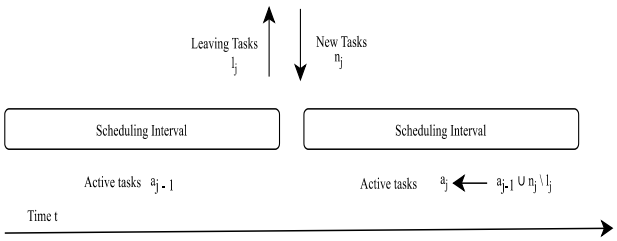

Fig .2 Workload Model

The intervals, shown in Figure 2, are numbered in sequence according to when they occur. The scheduling interval commences at time t and spans until the commencement of the subsequent interval. The active tasks refer to the tasks that are currently being executed on the hosts. Moreover, at the start of SP_j d, certain finished jobs are represented as d, while the newly assigned tasks through the WGM are labelled as n_j.. From the system, Tasks are removed, while fresh ones are added. Therefore, at the beginning of the period, the tasks now being dealt with are managed.

Table 1: Symbol Table

Symbol	Meaning
SP_j	j^{th} scheduling period
n_j	New tasks received at beginning of SP_j
a_j	Active tasks in SP_j
n	Number of hosts in the Edge-Cloud Data centre
l_j	Tasks leaving at beginning of SP_j
{T}	Host assigned to task T
Hosts	Set of hosts in the Edge-Cloud Data centre
$Action_j^{PG}$	Scheduling decision at start of SP_j
T_j^U	j^{th} task in an enumeration of U
$Loss_j^{PG}$	Loss function for the model at start of SP_j
h_j	j^{th} host in an enumeration of U
m_j	Migratable tasks in a_i
FV_j^U	Feature vector corresponding to U at SP_j

3.3 Problem formulation

The present interest mainly centres on the effectiveness enhancing of the edge cloud scheduler, as further provided in Section 3, with respect to the varying workload discussed in the same section. The scheduler's performance is assessed by quantifying a statistic known as Loss, which is determined for each scheduling period. A lower Loss number indicates a scheduler that is more efficient. The variable 〖Loss〗_j denotes the efficiency of the scheduler during the jth interval. The group of "hosts" in the edge cloud environment is listed as [H_0, H_1,..., H_n]. The overall count of hosts that are present at any one-time during execution is considered to be n. Each task T assigned to a host is represented as {T}. The scheduler functions as an intermediary between the present condition of the system and the required actions, such as allocating hosts for new activities and making decisions on the migration of active processes. The starting state of SP_j, denoted as $State_j$, comprises the parameter values of Hosts, the active tasks that were not completed in the preceding interval (a_j - 1\l_j), and the new tasks (n_j).

During each iteration of a task grouping, denoted as a_j, the scheduler is responsible for determining whether each individual job should be assigned to a host or transferred. This grouping combines the previous set of tasks, $(a_j - 1\backslash l_j)$, with newly introduced tasks, n_j, while omitting any tasks that have been removed in set l_j. The specific designation for this determination is $Action_j = \{h \in Hosts\ for\ task\ T \mid T \in m_j \cup n_j\}$, which pertains to the task grouping a_j. Thus, the synthesized $Model$ problem is shown in equation 1.

Minimize $\sum_j Loss_j$

$Model$ Subject to $\forall j, Action_j = Model(State_j)$

$\forall j \forall T \in m_j \cup n_j, \{T\} \leftarrow Action_j(T)$ (1)

3.4 Reinforcement learning system

Here, we introduce a customised model for Reinforcement Learning that is specifically tailored to tackle the challenge stated in Section 3.3, with a specific emphasis on policy gradient learning. Firstly, we provide a detailed description of the technical specifications of the Neural Network. Then, we explain how Loss is integrated into our suggested framework. Within our specific framework, the A3C paradigm designates the actor component as responsible for both decision-making and executing actions, considering the present environmental situation. This actor represents the policy by specifying the probability distribution of various actions for a given state. Experiences are accumulated throughout time, resulting in commensurate modifications to the model. A3C introduces asynchrony by executing many agents simultaneously, with each agent independently exploring the environment. This configuration compares the actual reward received with the expected reward predicted by the critique. It demonstrates the agents' capacity to navigate the state space more effectively; therefore, accelerating the learning process for our model is illustrated in Figure 3.

The scheduler model receives the state as an input, that contains various host parameters. These characteristics include the utilisation and size levels of the bandwidth, CPU, disc space, and RAM. In addition, the State encompasses power characteristics, time-based cost per unit, million instructions per second (MIPS) for each host, response time, and the allocation of jobs to individual hosts. Hosts vary in terms of their processing power, memory capacity and availability of I/O resources (disc and bandwidth). Given that jobs within an edge-cloud framework have limitations on computation, memory, and Input / output operations, these factors are important in making informed scheduling decisions. In addition, enhancing the allocation of different jobs on a limited host cluster can enhance energy efficiency by putting inactive hosts into slumber. Hosts that have improved I/O capabilities, namely in terms of disc read/write rates, accelerate the efficient completion of I/O-intensive

operations and help prevent violations of Service Level Agreements (SLAs).

During the initial phase of interval S, the model is tasked with allocating a host to each task, taking into consideration the input State. The final output, known as an Action, involves allocating a host to each new job in $\in n_j$ and determining migration decisions for the ongoing tasks from the prior interval. The allocation must comply with feasibility requirements, guaranteeing that each migratable job can be transferred to the chosen host without suffering simultaneous migration. Furthermore, it is essential to avoid host overloading after assigning a host (h) to a task (T), ensuring that the Host h is appropriately matched to task T. Hence, the precise details of the Action are outlined in equation 2.

$$Action_i = \begin{cases} h \in Hosts \ \forall \ t \ \in n_j \\ h_{new} \in Hosts \ \forall \ t \ \in m_j \ if \ t \ has \ to \ be \ migrated \end{cases} \quad (2)$$

Within the framework of neural networks, the output can be represented as a vector that signifies the distribution preferences for each host with respect to each task. Instead of choosing One host is assigned to each activity, the prototypical produces a list of Hosts sorted in order of preference. This unregulated action does not have a direct effect on the process of modifying the assignment of tasks to hosts. Choosing the most appropriate host for each work is of utmost importance, particularly for tasks that can be migratable. We employ a penalty strategy for distinct actions done by our model:

Migration penalty: The migration failure rate is determined by dividing the number of tasks that the prototypical intended to migrate but could not migrate by the total no. of tasks and expressing it as a percentage.

Allocation penalty: For every individual task, determine the no. of hosts that were not able to be assigned to the tasks, even though they received a

In our learning model, the penalty is represented by the equation

$$Penalty_{j+1} = \frac{\sum_{t \in a_j} H^k = Action_j(t)^{H^k \in Action_j^{PG}(t)}}{a_j \times n} + \frac{\sum_{t \in a_{j-1} \setminus l} 1(t \notin m_j \wedge Action_j(t) \neq \{t\})}{a_j} \quad (3)$$

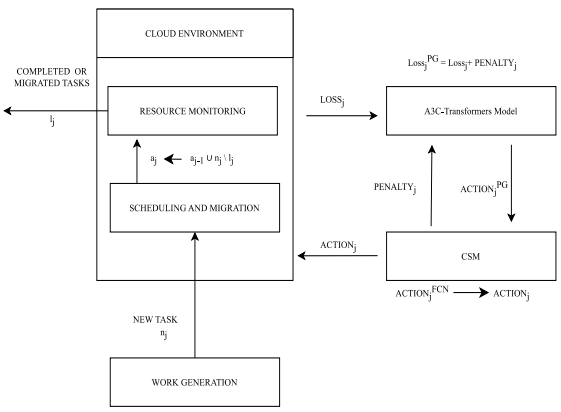

Fig. 3 learning model

3.5 Loss function

Our goal is to optimise the model to minimise the $Loss_j$ in each interval, hence minimising the overall cumulative loss. Furthermore, it is necessary for our model to be able to dynamically adjust to the changing states. The model functions as a mapping between $State_j$ to $Action_j$. In order to achieve this, we introduce the $Loss_j$ as a metric to direct parameter updates for this precise model.

Average energy consumption: The average energy consumption within a certain time is determined by the energy usage of the entire infrastructure, including both edge and cloud hosts. This consumption is then adjusted by dividing it by the greatest power output of environment. It is crucial to acknowledge that edge-and-cloud nodes may depend on distinct energy platforms, as indicated in Equation 4, examples include energy harvesting devices used to power edge nodes and serve as the primary power supply for cloud nodes. Therefore, the energy utilised at each host (h ∈ Hosts) is multiplied by the factor αh ∈ [0,1] that can be modified based on user preferences and deployment strategies for edge and cloud nodes.

$$AEC_j^{Hosts} = \frac{\sum_{h \in Hosts} \alpha_h \int_{t=t_j}^{t_{j+1}} P_h(t)dt}{\sum_{h \in Hosts} \alpha_h P_h^{max}(t_{j+1} - t_j)} \quad (4)$$

Average response time: The Average Response Time within an interval, represented as S_j, is the mean response time for all departing tasks (l_j+1) inside the intervals. Equation 5 demonstrates that this metric is normalised based on the highest response time observed within the current interval. The response time for completing a task is determined by adding the host's response time when the task is scheduled and the execution duration of the task.

$$ART_J = \frac{\sum_{t \in l_{j+1}} ResponseTime(t)}{l_{j+1} \, max_j max_{t \in l_j RespomseTime(t)}} \quad (5)$$

Average migration time*:* The Average Migration Rate within an interval, represented as S_j, is the total duration of migration for all currently active tasks (a_j), within that interval. Equation 6 describes how this measure is normalised by the maximum migration time up to the present interval.

$$AMT_j = \frac{\sum_{t \in a_j} MigrationTime(t)}{a_{j max_j max_{t \in l_j} ResponseTime\,(t)}} \quad (6)$$

Cost (C): The cost inside an interval, represented as SI_j, the total amount of money spent during a specific period, as mentioned in Equation 7.

$$Cost_j = \frac{\sum_{h \in Hosts} \int_{t=t_j}^{t_{j+1}} C_h(t)dt}{\sum_{h \in Hosts} C_h^{max}(t_{j+1} - t_j)} \quad (7)$$

The Average SLA Violations (SLAV): Within a given interval, the metric SI_j represents the mean number of SLA violations for departing tasks (I_{j+1}), as indicated by the SLA(t) of task T in Equation 8. This statistic is calculated based on two factors: (i) the amount of time SLA breaches occur for each active host, and (ii) the decrease in performance caused by migrations.

$$SLAV_j = \frac{\sum_{t \in l_{j+1}} SLA(t)}{l_{j+1}} \quad (8)$$

3.6 Dataset

The dataset used in this study is obtained from the Bitbrain dataset (Siqi Shen et al, 2015), this includes authentic data on measurements of resource consumption from important occupational workloads hosted on the Bitbrain system. The collection consists of logs from more than 1000 virtual machines (VMs) that are executing workloads on two different setups. This enables the measuring of expenses in real-time and in the evaluation of our model's success. The cost structure of the cloud layer is derived from the Microsoft Azure Infrastructure as a Service (IaaS) cloud service. The hourly costs in U.S. dollars are derived based on the cost of machines on comparable systems offered by Microsoft Azure (L. Roselli et al., 2015). The dataset has the subsequent attributes:

1. *Timestamp: Denoting the no. of milliseconds*
2. *CPU cores: Providing the no. of virtual CPU cores allocated*
3. *CPU capacity provisioned (CPU requested): The capacity of the CPUs can be determined by calculating the product of the number of cores and the speed per core, measured in MHz*
4. *CPU usage: Expressed in megahertz (MHz)*
5. *CPU utilisation percentage: It is a utilisation of the central processing unit (CPU) in the form of a percentage.*
6. *Memory provided (memory requested): Indicating the amount of memory in the virtual machine in kilobytes (KB).*
7. *Memory usage: Expressing the amount of memory currently being utilised in kilobytes.*
8. *Disk read throughput: Expressed in kilobytes per second (KB/s).*
9. *Disk write throughput: Expressed in kilobytes per second (KB/s).*
10. *Network received throughput: Expressing the rate of data transfer as kilobytes per second (KB/s).*
11. *Network transmission throughput: Expressed in kilobytes per second (KB/s).*

Next, we will proceed to develop the neural network model including the penalty design specified in Equation 9. To preserve Quality of Service (QoS) and minimise loss, it is essential to include penalties in the neural network model to ensure that every constraint is met. The host parameters utilised in this model are delineated in Table 2.

$$Loss_j^{PG} = Loss_j + Penalty_j \quad (9)$$

3.7 Policy Gradient Learning for Dynamic Scheduling

The specifications utilised for the simulation are provided in the Table 2. The entire system is comprised of the following components: At the beginning of each scheduling period, the Resource Management Service (RMS) receives task requests that include requirements for compute, bandwidth, and Service Level Agreement (SLA). The inputs, along possessing host attributes from the Resource Monitoring Service, allow the Deep Reinforcement Learning (DRL) model to make well-informed scheduling decisions. Afterwards, the Constraint Satisfaction Module determines feasible migration and scheduling choices using the output of the DRL model. The RMS enables users or IoT devices to immediately send requests to specific edge or cloud device assigned to the task, while efficiently handling additional tasks. Furthermore, a loss function is defined for Deep Reinforcement Learning technique, and its parameters are modified inside the environment. Prior studies have investigated the modelling of the $State_j$ to $Action_j^{PG}$ function to reduce the function $LOSS_j^{PG}$ using Q-Tables or neural network function approximators. Although these strategies are efficient, they provide deterministic policies that lack flexibility in unpredictable circumstances. On the other hand, our strategy focuses on directly estimating and improving the policy using policy gradient techniques, utilising $LOSS_j^{PG}$ as a guiding signal for updating the network.

Table 2. Hosts configuration

Processor	Cores	Name	RAM	MIPS	DISK BANDWIDTH	NETWORK	COST
EDGE LAYER							
Intel i5 3.2 GHz	4	DEPO X340H	16 GB	2000	49 MB/s	1 GB/s	0.23 $/hr
Intel i3 3.0 GHz	2	HITACHI HA 8000	8 GB	1800	76 MB/s	0.1 GB/s	0.11 $/hr
CLOUD LAYER							
Intel Xeon 2.3 GHz	64	DELL PE C6320	64 GB	2660	1024 MB/s	1.5 GB/s	6.94 $/hr
Intel Xeon 2.6 GHz	32	DELL PE R820	48 GB	1800	49 MB/s	1 GB/s	3.47 $/hr

3.8 Transformers Neural Network Architecture

The transformer architecture incorporates self-attention mechanisms, which allow the model to assess the significance of different segments in the input sequence while producing predictions. A typical transformer layer typically consists of many self-attention heads and feedforward neural networks. This structural structure enables the model to effectively capture linkages and interdependencies throughout the input sequence.

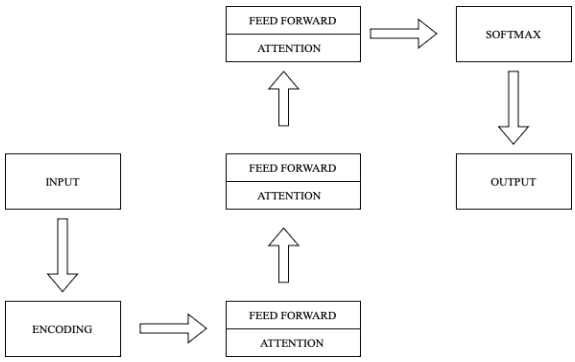

Fig. 4 Mechanism for encoding and decoding

The A3C learning algorithm integrates the actor-critic paradigm into the field of reinforcement learning. This framework consists of 2 main components: the critic, who evaluates the selected actions and provides feedback, and the actor, who is responsible for decision-making and action selection. Asynchronous updates are used to provide simultaneous learning across several actors, hence enhancing the learning speed of the model.

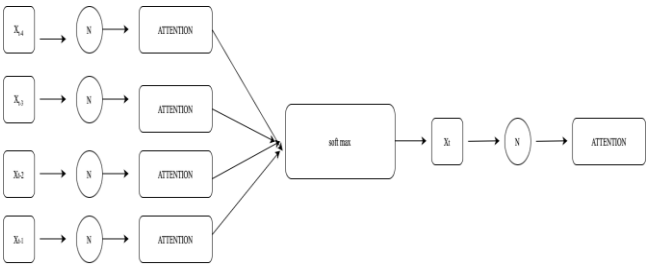

Fig. 5Feed forward in architecture

The integrated architecture combines transformer layers to represent and understand input sequences, along with the A3C learning algorithm to make decisions and optimise based on reinforcement signals. This technique combines the characteristics of both architectures to address complex and changing scheduling difficulties in cloud computing settings. The transformer architecture has a self-attention mechanism, which allows the model to analyse various aspects of the incoming data while producing predictions at specific time intervals. This approach improves parallel processing and considers the overall context, enabling the model to develop a thorough comprehension of the connections between various inputs to optimise resource utilisation efficiently. Fig 4, The neural network efficiently learns to make resource allocation decisions by collecting temporal relationships and interactions within workload data. The model includes the time-related aspects of task and host attributes, such as the CPU, RAM, and bandwidth requirements for tasks, as well as the corresponding capacities of hosts. The model can effectively make scheduling decisions by identifying temporal patterns, Fig.5. The feedforward network is used within the neural network layers of transformers to enhance the effectiveness of parallel task processing in architecture.

Feed Forward - The below SoftMax function serves as an activation function.

$$\sigma(\overrightarrow{z})_j = \frac{e^{z_j}}{\sum_{p=1}^{K} e^{z_p}} \tag{10}$$

The SoftMax function plays a vital role in transformers, as shown in Equation 10, by allowing the model to allocate weights to inputs according to their importance in the present job. This feature improves the model's capacity to focus on relevant information and understand the relationships and connections within the given data.

1.1 Policy learning

To train the Transformers model more effectively, the technique of back-propagation is utilised, where the reward is represented as $-Loss_j^{PG}$. The model incorporates an adaptive learning rate, which is initially set to10^{-3}) and is decreased to One-tenth when the sum of total of reward changes over the most recent ten iterations drop below 0.1. Automatic Differentiation is employed to adaptively update network parameters based on the reward denoted as $-Loss_j^{PG}$. Gradients originating from edge nodes hosting local network are accumulated in a manner that is not synchronised or happening at the same time. Periodically contributing to the alteration of global network properties. The updating technique employs a blend of gradients and Mean Square Error to minimise the anticipated cumulative loss. Within each scheduling interval, the Transformers model executes a forward pass using a batch size of 12 episodes. The model parameters are updated at regular intervals of 12. Algorithm 1 outlines the process of updating and scheduling the model via back-propagation. The complexity study reveals that the temporal complexity for N scheduling intervals is O(abN), with O(ab) being the dominant factor due to pre-processing and the production of $Action_j$ and $Penalty_j$.

Figure 6 and Figure 7 demonstrate how transformers work by using encoders, normalisation, and the self-attention mechanism. This method allows the model to allocate various weights to distinct portions of the input sequence based on their relevance. By employing this approach, the model is able to evaluate the context of each individual token in relation to all other tokens, thereby accurately capturing the dependencies across the sequence.

The gradients computed by local networks at each individual edge node are collected asynchronously, and the worldwide network parameters are updated regularly, using the methods explained in (V.Mnih et al., 2016). The rule for gradient buildup after the j^{th}scheduling interval is expressed, in accordance with the method outlined in equation 11. In this instance, θ refers to the general characteristics of the network, whereas θ' reflects the parameters of the local network (where a single gradient is applied because there is only one network with two heads).

$$d\theta \leftarrow \mathrm{d}\theta - \alpha\nabla_{\theta'} log[\pi(State_j : \theta')](Loss_j^{PG} + CLoss_{j+1}^{P_{red}}) + \alpha\nabla_{\theta'}(Loss_j^{PG} + CLoss_{j+1}^{P_{red}} - CLoss_j^{P_{red}})^2 \tag{11}$$

To decrease this number, the gradients are scaled proportionally with a negative sign to minimise the overall damage. The second gradient factor incorporates the Mean Square Error (MSE), which quantifies the discrepancy between the anticipated cumulative loss and the subsequent one-step ahead cumulative loss. The output obtained, denoted as $Action_j^{PG}$, is transformed into $Action_j$by the Constraint Satisfaction Module (CSM) and then transmitted to the Resource Management Service (RMS) during each scheduling interval. Therefore, a forward pass of the transformer network takes place at regular intervals. We employ a batch size-of twelve, where we retain the knowledge from the earlier interval to build gradients. These gradients are then used to update the model parameters after a succession of 12 intervals. The ideal episode size is determined through empirical analysis, considering that bigger batch sizes result in slower parameter updates, while

smaller batch sizes can provide difficulties in gradient accumulation due to increased variance.

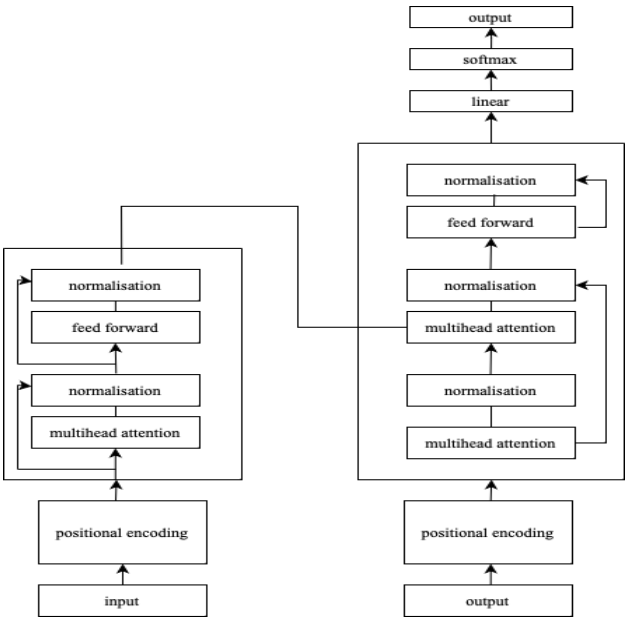

Fig .6 Operational principles of transformers

In this simulated scenario, the scheduling interval is set at 5 minutes. This implies that the feedforward process is carried out once every simulated hour, which corresponds to 12 periods. The algorithm concisely describes the iterative model iteration and scheduling procedure, which incorporates feedforward and attention mechanisms. Through systematic pre-processing and submission of the time interval's state to the transformers model, the network parameters are adjusted by incorporating losses and penalties. This approach allows the model to adapt flexibly to different environmental and application needs.

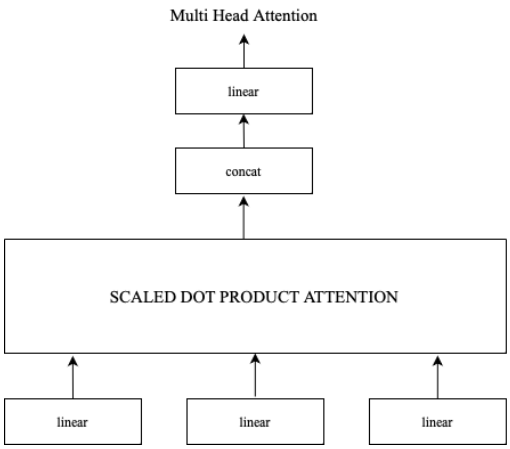

Fig 7 Multi head attention mechanism

Complexity Analysis: The Algorithm's complexity is modified by many tasks. The pre-processing of the input state has a time complexity of O(ab), where a × b denotes the maximum size of any feature vector among V_j^{Hosts}, $FV_j^{n_j}$,$FV_j^{a_j-1\backslash l_j}$. The computation of $Action_j$and $Penalty_j$takes O(n) time, considering the n hosts and tasks. Considering that the feature vectors possess a larger size than the no. of hosts, the complexity is O(ab) instead of O(n^2). With the exclusion of backpropagation and the forward pass, which are performed on GPUs, the time complexity for N scheduling intervals can be expressed as O(abN).

Algorithm 1. Dynamic Scheduling Pseudo Code

Inputs:

The value of N represents the total number of scheduling intervals.

batch size B

Begin

for *interval index j from 1 to N do*

if *j is greater than 1 and j is divisible by B then*

Use $Loss_j^{PG} = Loss_j + Penalty_j$ *in RL Model for back-propagation*

end if

Send State for preprocessing to RL Model

probabilityMap< output of RL Model for State

$(Action_j, Penalty_{j+1})$ *- CONSTRAINTSATISFACTIONMODULE (probabilityMap)*

Allocate new tasks and migrate existing tasks based on $Action_j$

Execute tasks in edge-cloud infrastructure for interval SP_j

end for

END

4. Result Evaluation and Performance Evaluation

This section will provide a description of the experimental setup used and evaluate the proposed model against baseline algorithms to compare their performances.

4.1 Experimental Setup

The focus of our study is to assess a scheduling framework based on Deep Learning. This required us to create a simulation environment that is tailored to our needs. This was achieved by enhancing the capabilities of CloudSim toolkits and iFogSim, both of which now provide resource monitoring services. Our simulation implementation followed the rules specified in Section 3.7 by separating the execution into regular scheduling intervals. In order to ensure consistency with prior studies, we chose to use a 5-minute interval period. This allowed us to directly compare our results with the baseline methods.

In accordance with iFogSim's principles, the Cloudlets, which represent the jobs, were created using the WGM method and the Bitbrain dataset (Siqi Shen et al., 2015). The integration of iFogSim and CloudSim allowed for the incorporation of variable characteristics, such as reaction time, power, and cost attributes, into the edge unit nodes. Furthermore, innovative modules were developed to imitate the mobility of IoT devices, incorporating characteristics such as variations in bandwidth, delays in task completion, and integration with deep learning technique. Supplementary software components were created for the constraint satisfaction module (CSM), input pre-processing, and change

of output. In addition, our study examines two prevalent forms of Reinforcement Learning (RL) approaches commonly seen in the literature:

LR-MMT: It is a workload scheduler that combines local regression (LR) with minimum migration time (MMT) heuristics to identify overloaded conditions and choose tasks. This approach is described in full in reference (A. Beloglazov et al., 2012).

MAD-MC is a dynamic scheduler that utilises the median absolute deviation (MAD) and maximum correlation policy (MC) to schedule workloads. It is designed to detect system overload and assign tasks based on these policies. This approach is discussed in detail in reference (A. Beloglazov et al, 2012).

DDQN Model: It is an implementation of the optimised Double Deep Q-Network (DQN) methodology, which is a conventional Deep Q-Learning method that has been the focus of several studies, including references (D. Basu, 2019).

DRL (REINFORCE): this approach uses the REINFORCE method, which is based on policy gradients, and employs a fully connected neural network architecture. This method notably circumvents asynchronous updates and recurrent network configurations.

Furthermore, to enhance our comparative analysis, we have included an extra baseline method in our study:

A3C-R2N2: The A3C learning algorithm integrates a residual recurrent neural network architecture. This technique provides a refined and specialised perspective, specifically tailored for our experimental framework.

It is crucial to highlight that these algorithms have been tailored to specifically address our unique problem domain, guaranteeing a comprehensive and contextualised evaluation in comparison to our offered models. In order to ensure fairness in the comparison, the RL model is designed to match a state representation that closely resembles "$State_j$" as described in Section 3.4. The actions in this paradigm represent the movement from one state to another within a periodic state space. Following approaches described in reference (Zhenyu Shi et al), the DQN Model diverges from Bellman's Equation (Q. Zhang et al, 2017) assigning rewards in $Loss^{PG}$. The adoption of the REINFORCE approach eliminates the asynchronous updates or recurrent network designs in order to maintain consistency in evaluation criteria.

4.2 Evaluation with Baseline Algorithms and Justifications

Evaluation with baseline Algorithms is as illustrated below.

1. Energy Consumption - Transformers are characterised by their capacity to perform parallel processing, which sets them apart from sequential models such as R2N2 that have inherent constraints. The use of Transformers allows for the efficient allocation of hardware resources. Transformers possess an innate self-attention mechanism that enables them to simultaneously grasp dependencies among different elements. The presence of parallelism has the capacity to reduce the total amount of energy used, as demonstrated in Figure (a).
2. Response Time and SLA Violations - As illustrated in Fig. (b), the Transformer model demonstrates the ability to process input sequences concurrently, which expedites computation and reduces response times. Enhanced efficiency facilitates the effective fulfilment of Service Level Agreements (SLAs) through the more streamlined management of tasks, as illustrated in Figure (c). On the other hand, R2N2 models function in a sequential fashion, which may result in extended response times and compromise service level agreements (SLAs) for situations that require prompt responses.
3. Total Cost for Resources - Transformers' parallel architecture facilitates the maximisation of resource utilisation. By employing self-attention and encoders, it is possible to optimise resource utilisation, which may result in a decrease in the overall expenses associated with computational resources, as illustrated in Figure (d). Conversely, R2N2 models may necessitate more resources as a result of their sequential processing methodology, thereby augmenting the overall expenditures.
4. Average Task Completion and Total Completed Tasks - The parallel processing capabilities of Transformers leads to faster task completion, as shown in Figure (e), and a higher overall number of tasks performed within a certain timeframe, as indicated in Figure (f). Transformers are enhanced with a self-attention mechanism to improve their ability to process input sequences. On the other hand, R2N2 models typically demonstrate extended durations for completing tasks.
5. Task Migration and Total Migration Time - The Transformers model is the most effective deep reinforcement learning (DRL) method for dynamic scheduling in edge-cloud computing. This is because Transformers have the capability to efficiently manage parallel jobs, thanks to their self-attention mechanism and encoders. Additionally, Transformers can accurately capture long-range relationships in sequences. This capacity can be beneficial in dynamic scheduling settings, potentially decreasing the necessity for frequent task migrations, as demonstrated in Fig (g) and Fig (h).

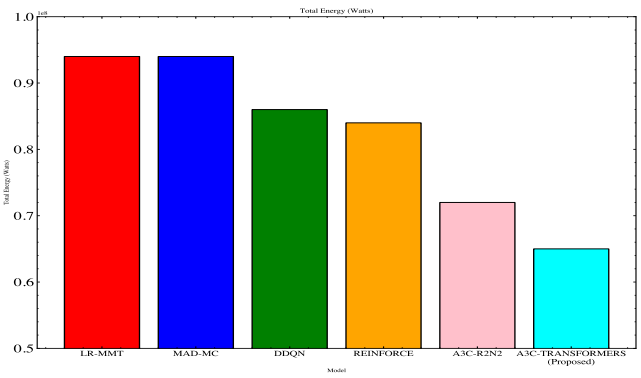

Fig (a) Cumulative energy Consumption

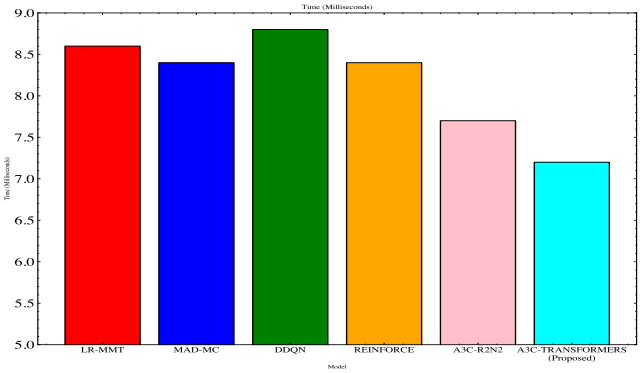

Fig (b) Mean Response Time

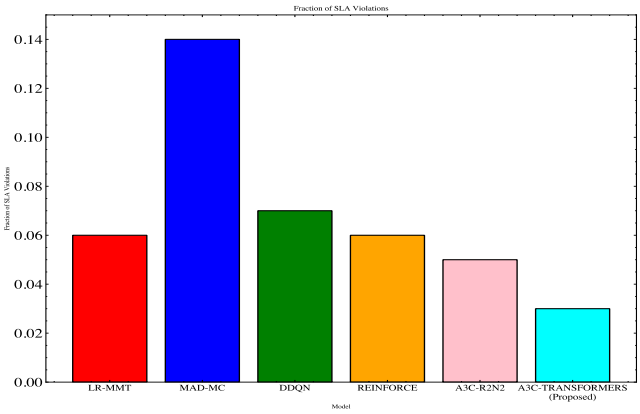

Fig (c) SLA Offences

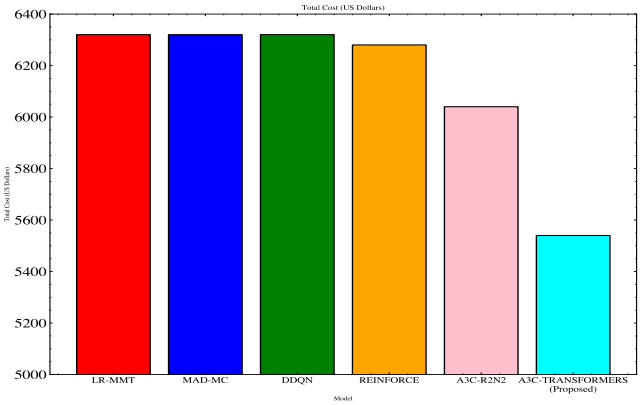

Fig (d) Overall Expense

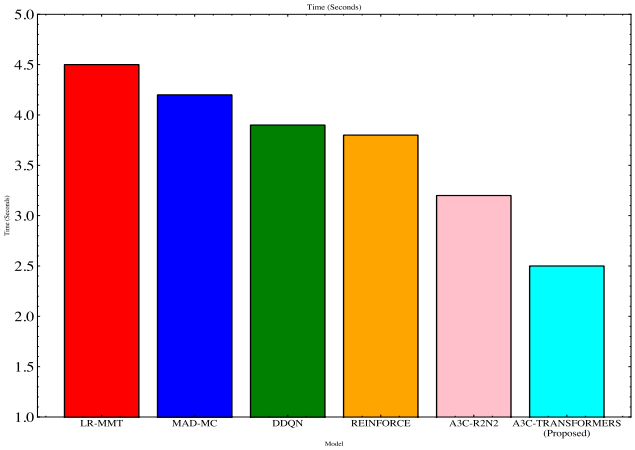

Fig (e) Average Task Completion

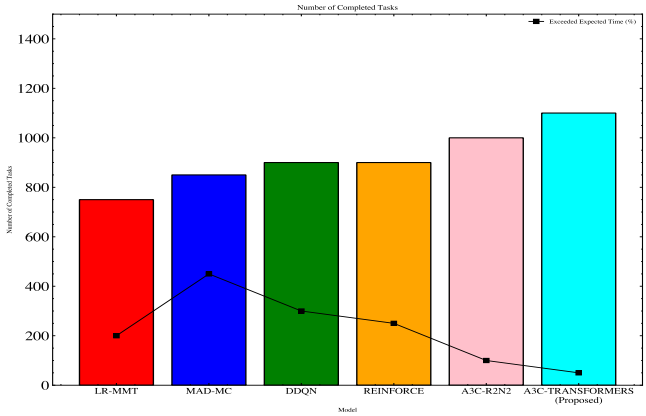

Fig (f) Total Completed Tasks

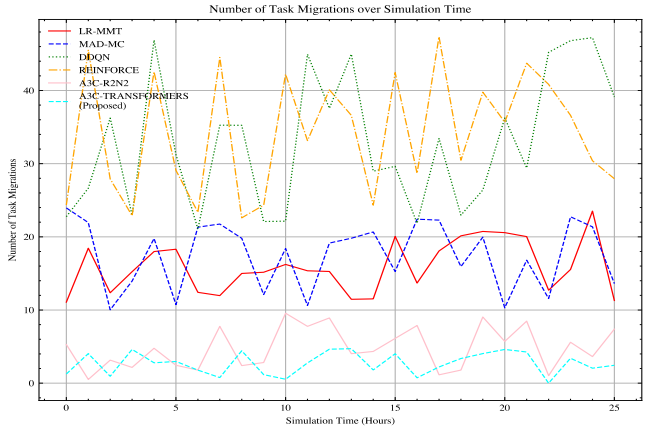

Fig (g) Migration of tasks at regular intervals

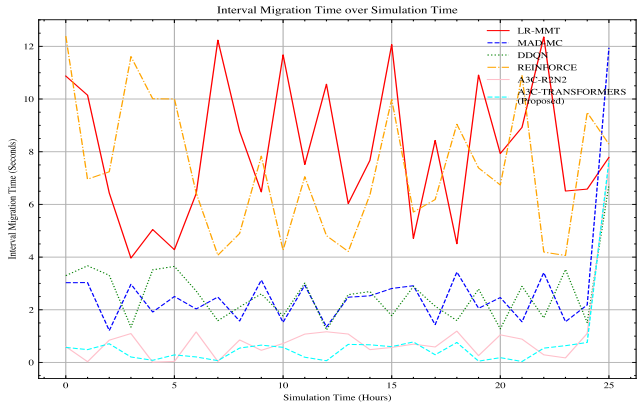

Fig (h) Duration of Migration in Each Interval

4.3 Performance Comparison

REINFORCE and A3C-R2N2, compared to other conventional algorithms, demonstrate comparatively lower levels of energy usage. Nevertheless, our A3C Learning model and Transformer-based model outperform them, demonstrating a decrease of 22.4% and 9.7% respectively compared to REINFORCE and A3C-R2N2, as specified in Table 3 and Table 4. The superiority of our approach arises from its capacity to adapt to various job workloads. It efficiently assigns resource-intensive tasks to strong machines and strategically reduces the number of active hosts through energy-efficient scheduling decisions. In addition, our model demonstrates a considerable improvement in average response time compared to the baseline algorithms,

with a 6.4% enhancement over the most effective baseline (A3C-R2N2). Our methodology optimises task distributions by considering node classifications (edge or cloud) and minimising work migrations. This leads to a significant 40% reduction in SLA violations, ensuring efficient performance. The cost-effectiveness of our approach is evident, as the A3C Learning model and Transformer-based model have the lowest data centre costs. This is achieved through optimised task allocation and the use of cost-efficient cloud Virtual Machines (VMs), resulting in an 8.3% reduction compared to A3C-R2N2.

Our model excels in terms of average task completion time, demonstrating a 21.8% decrease compared to the most efficient baseline. Moreover, it demonstrates exceptional rates of work completion within designated timeframes, highlighting its ability to minimise the negative impacts of migrations on task performance. Despite a slight increase in scheduling overhead (0.001%) due to frequent changes of the Transformer architecture, the significant performance improvements surpass this minor increment. Our technique establishes itself as a high-level scheduler in dynamic Edge-Cloud contexts characterised by unpredictable workloads.

Table 3: Comparative analysis of several methodologies

Method	✕	✕	✓	✕	✕	✕	✕
HEURISTICS	✕	✕	✓	✓	✓	✕	✕
GAUSS PR	✕	✕	✓	✓	✓	✓	✕
DQN	✕	✕	✓	✓	✓	✓	✕
Q LEARNING	✕	✕	✕	✓	✓	✕	✕
DNN	✕	✕	✓	✓	✓	✓	✕
DDQN	✕	✕	✓	✓	✓	✓	✕
DRL(REINFORCE)	✓	✓	✓	✓	✓	✓	✓
DRL(R2N2)	✓	✓	✓	✓	✓	✓	✓
DRL (A3C-Transformers) (Proposed)	✕	✕	✓	✕	✕	✕	✕

Table 4: Comparative analysis of several methodologies

Method	Energy	Response Time	Sla Violations	Cost	Self-Attention
HEURISTICS	✓	✕	✓	✕	✕
GAUSS PR	✓	✕	✓	✕	✕
DQN	✓	✕	✕	✓	✕
Q LEARNING	✓	✕	✕	✓	✕
DNN	✕	✕	✓	✓	✕
DDQN	✓	✕	✕	✕	✕
DRL(REINFORCE)	✕	✓	✕	✕	✕
DRL(R2N2)	✓	✓	✕	✓	✕
DRL (A3C-Transformers) (Proposed)	✓	✓	✓	✓	✓

6. Conclusion and Future Scope

Effectively handling resources in dynamic Edge-Cloud contexts is a substantial difficulty, particularly when considering unpredictable workloads and the many characteristics of these intricate systems. The study described in this paper is a significant breakthrough in the field as it introduces an advanced scheduling methodology that blends A3C Learning with Transformer designs. Our model outperforms existing methods, demonstrating substantial improvements in important performance metrics.

Our approach effectively improves scheduling efficiency in dynamic Edge-Cloud environments by significantly reducing energy consumption (9.7%), response time (6.4%), SLA violations (40%), and cost (8.3%). Additionally, it achieves a remarkable 21.8% enhancement in average task completion time.

Future Scope - Moving forward, several promising avenues exist to expand upon and refine the current model:

- Real-World Implementation: Similar to the base paper's future work, transitioning from simulation to real Edge-Cloud environments is paramount. Continuous profiling of CPU, RAM, and disk requirements for new tasks is essential for effective scheduling. Incorporating methodologies like exponential averaging of resource requirements and synchronised data collection across A3C agents will be pivotal.
- Scalability Analysis: Evaluating scalability under varying host and task loads is imperative. Exploring the potential of advanced reinforcement learning models, such as Impala, can enhance scalability while ensuring effective scheduling for diverse setups.
- Data Privacy and Security: Investigating and integrating robust measures for data privacy and security is crucial. Addressing potential vulnerabilities within Edge-Cloud environments will bolster trust and reliability.
- Further Performance Enhancements: Continuous refinement of the scheduling model to achieve even more substantial reductions in resource consumption, response time, and cost is an ongoing pursuit. Leveraging evolving technologies and architectures could yield additional efficiencies.

By delving into these future directions, the aim is to not only solidify the current model's applicability but also equal contribution for the advancement of scheduling algorithms in dynamic Edge-Cloud environments.

References

A. Beloglazov and R. Buyya, “Optimal online deterministic algorithms and adaptive heuristics for energy and performance efficient dynamic consolidation of virtual machines in cloud data centres,” *Concurrency and Computation: Practice and Experience*, vol. 24, no. 13, pp. 1397–1420, 2012.

Chang Lu, Jie Zhu, Haiping Huang, YuzhongSun,A multi-hierarchy particle swarm optimization-based algorithm for cloud workflow scheduling, Future Generation Computer Systems, Volume 153,2024,Pages 125-138,ISSN 0167-739X,https://doi.org/10.1016/j.future.2023.11.030.

D. Jeff, “ML for system, system for ML, keynote talk in Workshop on ML for Systems, NIPS,” 2018. [Online]. Available: http://mlforsystems.org/ .

D.Basu, X.Wang, Y. Hong, H. Chen, and S. Bressan, “Learn-as- you-go with megh: Efficient live migration of virtual machines,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 30, no. 8, pp. 1786–1801, 2019.

Ding, Z. *et al.* (2024). Accelerated Genetic Algorithm with Population Control for Energy-Aware Virtual Machine Placement in Data Centers. In: Luo, B., Cheng, L., Wu, ZG., Li, H., Li, C. (eds) Neural Information Processing. ICONIP 2023. Lecture Notes in Computer Science, vol 14448. Springer, Singapore. https://doi.org/10.1007/978-981-99-8082-6_2

G. Fox, J. Glazier, J. Kadupitiya, V. Jadhao, M. Kim, J. Qiu, J. P. Sluka, E. Somogy, M. Marathe, A. Adiga *et al.*, “Learning everywhere: Pervasive machine learning for effective high-performance computation,” in *IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 2019, pp. 422–429.

Guangyao Zhou Wenhong Tian Rajkumar Buyya Deep Reinforcement Learning-based Methods for Resource Scheduling in Cloud Computing: A Review and Future Directions https://doi.org/10.48550/arXiv.2105.04086.

H. Mao, M. Alizadeh, I. Menache, and S. Kandula, “Resource management with deep reinforcement learning,” in *Proceedings of the 15th ACM Workshop on Hot Topics in Networks*. ACM, 2016, pp. 50–56.

Hamid Reza Naji, Reza Esmaeili Reducing energy consumption of cloud data centers using proper placement of virtual machines.

J.Wang, K. Liu, B. Li, T. Liu, R. Li, and Z. Han, “Delay-sensitive multi-period computation offloading with reliability guarantees in fog networks,” *IEEE Transactions on Mobile Computing*, 2019.

L. Roselli, C. Mariotti, P. Mezzanotte, F. Alimenti, G. Orecchini, M. Virili, and N. Carvalho, “Review of the present technologies concurrently contributing to the implementation of the internet of things (iot) paradigm: Rfid, green electronics, wpt and energy harvesting,” in *Proceedings of the Topical Conference on Wireless Sensors and Sensor Networks (WiSNet)*. IEEE, 2015, pp. 1–3.

Lei Shi, Jing Xu, Lunfei Wang, Jie Chen, Zhifeng Jin, Tao Ouyang, Juan Xu, and Yuqi Fan authored the article titled "Multijob Associated Task Scheduling for Cloud Computing Based on Task Duplication and Insertion," published in Wireless Communications and Mobile Computing, volume 2021, with Article ID 6631752,13 pages. https://doi.org/10.1155/2021/6631752.

M. Bowling, “Convergence problems of general-sum multiagent reinforcement learning,” in *ICML*. MLR Press, 2000, pp. 89–94.

M. Cheng, J. Li, and S. Nazarian, “DRL-cloud: Deep reinforcement learning-based resource provisioning and task scheduling for cloud service providers,” in *Proceedings of the 23rd Asia and South Pacific Design Automation Conference*. IEEE Press, 2018.

M. Xu, S. Alamro, T. Lan, and S. Subramaniam, “Laser: A deep learning approach for speculative execution and replication of deadline-critical jobs in cloud,” in *Proceedings of the 26th International Conference on Computer Communication and Networks (ICCCN)*. IEEE, 2017.

M.J. Diván, D. Shchemelinin, M.E. Carranza, C.I. Martinez-Spessot, M. Buinevich Real-Time Reliability Monitoring On Edge Computing: A Systematic Mapping Informatics and Automation. 2023. Vol. 22 No. 6. ISSN 2713-3192 (print) 1243 ISSN 2713-3206 (online) www.ia.spcras.ru DOI 10.15622/ia.22.6.1.

McKinsey & Company, “The Internet of Things: How to capture the value of IoT,” May, 2018.

Microsoft Azure pricing calculator https://azure.microsoft.com/en-au/pricing/calculator/.

Minh-Ngoc Tran, YoungHanKim,Optimized resource usage with hybrid auto-scaling system for knative serverless edge computing,Future Generation Computer Systems, Volume 152,2024,Pages 304-316,ISSN 0167-739X,https://doi.org/10.1016/j.future.2023.11.010.(https://www.sciencedirect.com/science/article/pii/S0167739X23004156).

Moghaddasi, K., Masdari, M. Blockchain-driven optimization of IoT in mobile edge computing environment with deep reinforcement learning and multi-criteria decision-making techniques. *Cluster Compu*ting (2023). https://doi.org/10.1007/s10586-023-04195-4.

Q. Zhang, M. Lin, L. T. Yang, Z. Chen, and P. Li, “Energy- efficient scheduling for real-time systems based on deep q- learning model,” *IEEE Transactions on Sustainable Computing*, vol. 4, no. 1, pp. 132–141, 2017.

R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. De Rose, and R. Buyya, “Cloudsim: a toolkit for modelling and simulation of cloud computing environments and evaluation of resource provisioning algorithms,” *Software: Practice and experience*, vol. 41, no. 1, pp. 23–50, 2011.

R. Nithiavathy, SengathirJanakiraman, and M. Deva Priya have contributed to the article titled "Adaptive Guided Differential Evolution-based Slime Mould Algorithm-based Efficient Multi-objective Task Scheduling for Cloud Computing Environments," published with the DOI: https://doi.org/10.1002/ett.4902.

SanathMugeraya, Kailas Devadkar Dynamic Task Scheduling and Resource Allocation for Microservices in Cloud Journal of Physics: Conference Series IOP Publishing doi:10.1088/1742-6596/2325/1/012052.

Siqi Shen, Vincent van Beek, Alexandru Iosup, Statistical Characterization of Business-Critical Workloads Hosted in Cloud Datacenters, the 15th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid), 2015, ShenZhen, China. http://gwa.ewi.tudelft.nl/datasets/gwa-t-12-bitbrains.

T.Choudhari, M. Moh, and T.-S. Moh, “Prioritised task scheduling in fog computing,” in *Proceedings of the ACMSE Conference*, ser. ACMSE ’18. New York, NY, USA: ACM, 2018, pp. 22:1–22:8.

The Bitbrain dataset is available here: http://gwa.ewi.tudelft.nl/datasets/gwa-t-12-bitbrains

Tuli Shreshth&Ilager, Shashikant &Ramamohanarao, Kotagiri & Buyya, Rajkumar. (2020). Dynamic Scheduling for Stochastic Edge-Cloud Computing Environments Using A3C Learning and Residual Recurrent Neural Networks. IEEE Transactions on Mobile Computing. PP. 1-1. 10.1109/TMC.2020.3017079.

V.Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, “Asynchronous methods for deep reinforcement learning,” in *Proceedings of the International conference on machine learning*, 2016, pp. 1928–1937.

Vaswani, A. et al.: Attention is all you need. In: Advances in Neural Information Processing Systems. pp. 5998--6008 (2017).

Volume 24, Issue 4,2023,100418,ISSN 1110-8665,https://doi.org/10.1016/j.eij.2023.100418.

X. Chen, L. Jiao, W. Li, and X. Fu, “Efficient multi-user computation offloading for mobile-edge cloud computing,” *IEEE/ACM Transactions on Networking*, vol. 24, no. 5, pp. 2795–2808, 2015.

Z. Cao, H. Zhang, Y. Cao and B. Liu, "A Deep Reinforcement Learning Approach to Multi-Component Job Scheduling in Edge Computing," *2019 15th International Conference on Mobile Ad-Hoc and Sensor Network (MSN)*, Shenzhen, China, 2019, pp. 19-24, doi: 10.1109/MSN48538.2019.00018.

Zhenyu Shi, Tianhao Zhao, Qi Li, Zhixia Zhang, ZhihuaCui,Workflow migration in uncertain edge computing environments based on interval many-objective evolutionary algorithm,Egyptian Informatics Journal,

C. J. Watkins and P. Dayan, “Q-learning,” *Machine learning*, vol. 8, no. 3-4, pp. 279–292, 1992.

D.-M. Bui, Y. Yoon, E.-N. Huh, S. Jun, and S. Lee, “Energy efficiency for cloud computing system based on predictive optimization,” *Journal of Parallel and Distributed Computing*, vol. 102, pp. 103–114, 2017.